

1 · WHAT ELASTICSEARCH IS — AND THE ONE THING IT IS NOT

The definition
A distributed search and analytics engine built on Apache Lucene. Each shard is a Lucene index. Elasticsearch adds distribution, replication, a REST API and a query language on top. Understanding that Lucene is underneath explains most of its behaviour — including the parts that surprise people.

The core data structure is the **inverted index**. Rather than mapping documents to terms, it maps terms to the **documents containing it**. That is why full-text search over hundreds of millions of documents returns in milliseconds — and why the text must be broken into terms first, which is what analysis does and why mappings matter so much.

Near-real-time, started precisely
A document is **not searchable the instant it is indexed**. It becomes visible after a **refresh**, which creates a new searchable segment. The default interval is 1s.

The refinement most summaries omit: Indices that have received no search for a period become **search-Idle** and stop refreshing on that schedule — and why the refresh happens when a search next arrives. So “one second” is the busy-idle case, not a universal guarantee. Use `7 refresh=wait_1m` for when a write must be visible to the very next read.

The most important sentence on this sheet: **Elasticsearch is not a system of record**. There are no multi-document transactions, no rollback, no foreign keys, and no join in the relational sense. Visibility is delayed by design. It is a **search and analytics layer over data that lives somewhere else**, and it should always be rebuildable from that source.

The practical test: If losing the entire cluster means re-indexing from a database or an event log over some hours, the architecture is sound. If it means losing data that exists nowhere else, the design has a defect — and no replica count or snapshot schedule fully compensates for it.

The vocabulary, mapped to what it actually is
Term: What it really is
Index: A logical collection of documents with a shared mapping — not a database table, because the mapping is for less changeable
Document: A JSON object. The unit of indexing and retrieval
Mapping: The schema: field names, types and how each is analysed. **Largely immutable once set**
Shard: A **complete Lucene index**. The unit of distribution, and the unit of resource cost
Segment: An immutable file within a shard. Written by writer, combined by merges
Replica: A copy of a primary shard — availability and read capacity
Node / cluster: A JVM process, a group of them sharing cluster state

STAFF: The licensing history, which comes up. Elasticsearch was Apache 2.0, moved in 2021 to a dual SSPL and Elastic License 2.0 arrangement, then in September 2024 Elastic added AGPLv3 as an OSI-approved option — so it is now tri-licensed and once again open source by the OSI definition. QuidSearch was forked by AWS from 7.10.2 in April 2021 under Apache 2.0 and moved to the Linux Foundation in September 2024. The architecture on this sheet applies broadly to both, though features have diverged since the fork.

2 · MAPPINGS — THE SCHEMA YOU CANNOT CHANGE

You cannot change the type or analyzer of an existing field. The inverted index was built at index time using those settings — altering them would invalidate every term already written. Adding a new field is fine; changing an existing one requires **reindexing into a new index**. This is the single biggest difference from a relational schema, and planning for it is not optional.

text versus keyword — the most common mistake here

	text	keyword
Analysed?	Yes — broken into terms	No — stored whole
For	Full-text search, relevance	Exact match, filters, sorting, aggregations
Set / aggregate	No — needs fieldsets, which is expensive and off by default	Yes — via doc values
Exact match	Unreliable — the stored terms are analysed, so a term query usually fails	Exact

The default handles both, and it is worth knowing why. Dynamic mapping maps a string as text with a `keyword` sub-field. So title searches full-text and title, keyword sorts and aggregates. This is a **multi-field**, and it is why so much work without anyone defining a mapping — and why the aggregation on title that failed works when you do `_type.keyword`.

The classic confusion: **match versus term**, which analyses your query text the same way the field was analysed; term does not and looks for the literal value. A term query for “New York” against an analysed field returns nothing, because the index contains new and york separately.

Dynamic mapping, and where it turns dangerous

- The first document defines the field. Whatever type is inferred is what you are committed to
- A field arriving as “123T” once becomes a string forever, and a later numeric range-query on it will not work as expected
- Set dynamic: strict on anything important, so an unexpected field is an error rather than a silent commitment

Mapping explosion — a real cause of cluster outages. If field names come from user or customer data — an object keyed by identifier, or arbitrary log attributes — every new key becomes a new mapped field. The mapping grows without limit, the cluster state grows with it, and because cluster state is replicated to every node the whole cluster degrades. There is a default limit of 1000 fields per index, and hitting it is a symptom rather than the problem.

The fix is a modelling one: store such data as an array of key/value pairs — `{[k: "colour", v: "red"]}` — which is two fields regardless of how many distinct keys exist. The flattened field type solves the same problem, mapping an entire object as a single field at the cost of some query flexibility.

Changing a mapping without downtime

```
if applications always read and write through an ALIAS
for /calluser {select type
{ "name": "index", "orders-v1", "aliases": "orders" } } }
# 1. create orders-v2 with the corrected mapping
# 2. reindex, which re-analyses every document
root /reindex { "source": { "index": "orders-v1", "type": "order" }, "target": { "index": "orders-v2" } }
# 3. swap the alias immediately, then stop it
```

Never point an application at a concrete index name. Always use an alias. It costs nothing on day one and it is what makes every future mapping change a routine operation rather than an outage.

3 · TEXT ANALYSIS

how text becomes searchable terms

Analysis converts a text field into the terms stored in the inverted index. It runs in three ordered stages, and it applies **only to text fields** — never to keyword, numbers or dates.

1. CHARACTER FILTERS: strip HTML, replace chars
2. TOKENIZER: split into tokens (exactly one) — whitespace, stop words, stemming, synonyms
3. TOKEN FILTERS: filter, synonyms

What each stage contributes

- Char filter: Operates on the raw string before tokenising — removing markup, mapping characters
- Tokenizer: Exactly one, standard splits on word boundaries; keyword emits the whole input, ngram and edge_ngram produce partial-word tokens for autocomplete
- Token filters: Any number, in order. Lower case makes matching case-insensitive; stop removes common words; stemmer reduces to a root so running matches run; synonym expands equivalents

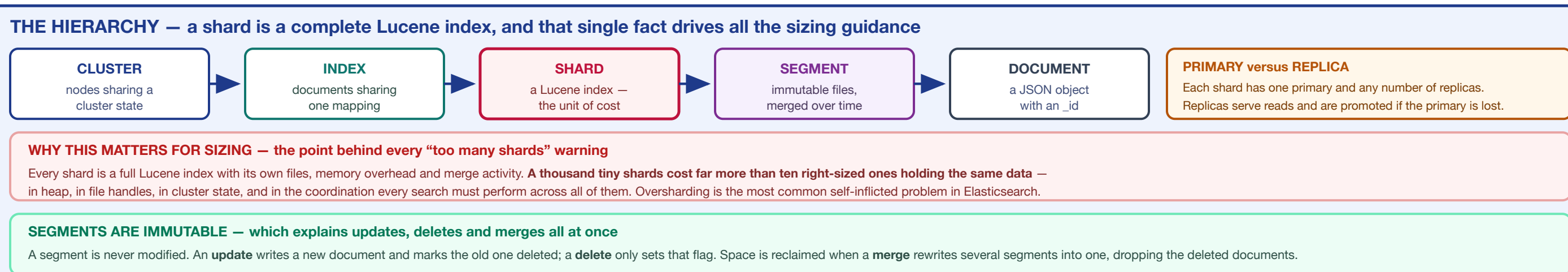
Index-time and query-time analysis must be compatible. The query text is analysed with the field’s search analyzer and the resulting terms are looked up. If the two chains disagree — different stemming, different case handling — **nothing matches**, and there is no error. It simply returns zero results, which is why this is hard to diagnose without the tool below.

The `_analyze` API is the debugging tool for exactly this. It shows the tokens a given analyzer produces, so you can compare what was indexed against what the query generates. It answers “why does this search return nothing” faster than anything else.

GET /orders/_analyze? {"index": "description", "text": "Running Shoes"} {"from": "from", "to": "to", "with": "with"} applied

Changing an analyzer requires a **reindex**, for the same reason a type change does — the existing terms were produced by the old chain. Design the analysis alongside the mapping, and use `edge_ngram` or the completion suggester for autocomplete rather than wildcard queries, which cannot use the index efficiently.

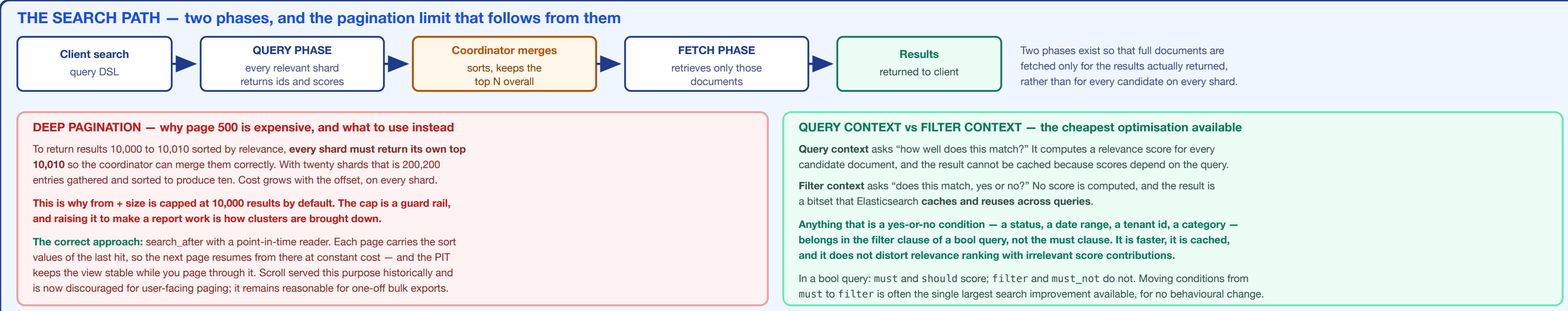
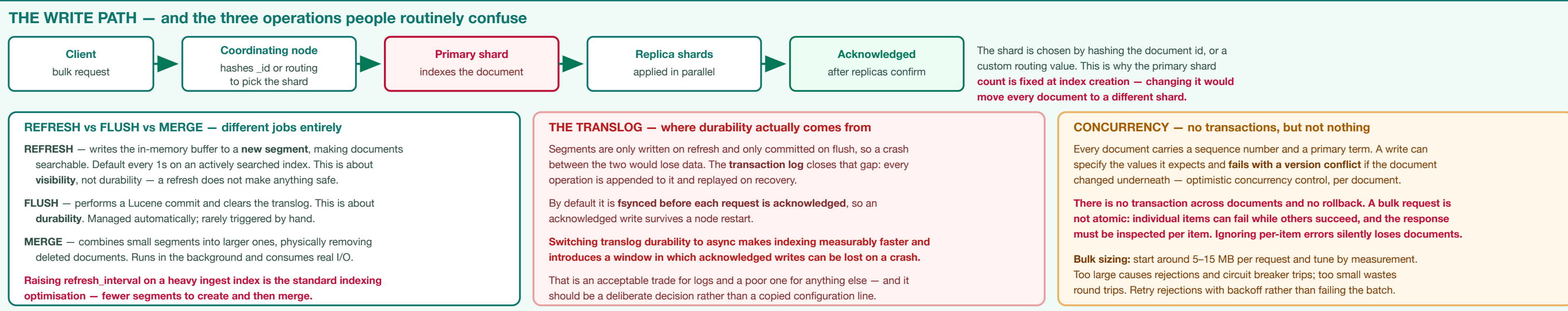
4 · ARCHITECTURE — NODES, SHARDS, SEGMENTS, AND THE TWO REQUEST PATHS



NODE ROLES — separating them is what makes a cluster stable at scale

- MASTER-ELIGIBLE: Maintains cluster state mappings, shard allocation rules. One is elected; a quorum of the voting configuration must agree. Use three dedicated masters in production — they do not do work
- DATA: Holds shards, executes queries and aggregations. Sub-roles by tier: hot, warm, cold and frozen, each with different hardware. The tier a shard sits on is driven by lifecycle policy — see page 10
- COORDINATING: Every node can coordinate. It receives a request, runs it and returns the result, and then the relevant shards, and merges the results before returning.
- INGEST, ML, TRANSFORM: Ingest nodes run pipelines that enrich or replace documents before indexing. ML and transform nodes run their own dedicated workloads. Separate them to a heavy pipeline control plane search

CLUSTER HEALTHY — three words that mean precisely one thing each
GREEN — every primary and every replica is assigned. Normal.
YELLOW — all primaries assigned, but one replica is not. No data is lost and searches still work, but redundancy is gone. A single-node cluster is permanently yellow.
RED — at least one primary is unassigned. Part of the data is unavailable right now, and searches touching it return partial results or fail. Investigate immediately.



READING THE DIAGRAM — the five properties that explain the rest of this sheet

- A shard is a whole Lucene index, and shard count is fixed at creation. This is why sizing must be decided up front, why overhauling is costly, and why growth is handled by rolling over to new indices rather than by resizing existing ones.
- Segments are immutable. Updates and deletes are additive, space is reclaimed only by merges, and merges consume I/O in the background. High update rates are therefore expensive in a way that is not obvious from the API.
- The mapping is written into the index at index time. Types and analyzers cannot be changed in place; reindexing behind an alias is the supported path, and designing for it on day one is what keeps schema changes routine.
- Search is scatter-gather across shards. Latency is set by the slowest shard, deep offsets multiply work on every shard, and more shards is not automatically faster — pass a point it is relatively slow.
- Visibility is delayed and there are no cross-document transactions. Elasticsearch is a search and analytics layer, not a system of record, and every design that forgets this eventually pays for it during a recovery.

Useful information of any Elasticsearch design: how large is a shard and how many are there, is every application field explicitly mapped, does logs go through an alias, are yes-or-no conditions in filter context, and can the whole cluster be rebuilt from source?

The last question is the one that matters most in an incident — and if the answer is uncertain, that uncertainty is the finding.

5 · INDEXING PERFORMANCE — SEGMENTS, MERGES AND THROUGHPUT

The levers, in the order worth trying

- Lower the heap and cost
- Use the bulk API: The single largest tip. Around 5-15 MB per request as a starting point, tuned by measurement. Never index documents one at a time at volume
- Raise refresh_interval: Fewer segments created and merged. **Costs visibility delay** — set to 30s for logs, or <1 during a bulk load and restore afterwards
- Index with zero replicas, then add: Only for an initial load into an empty index. No redundancy while it runs
- Let Elasticsearch decide its: A supplied id forces a check for an existing document; an auto-generated one skips it
- fewer, larger shards: Less coordination and merge overhead per document
- Map only what you query: Disable indexing on fields never searched, and doc_values on fields never sorted or aggregated
- Faster storage: Merging is I/O bound. Local SSD or NVMe makes a larger difference than most tuning

Do not use async translog durability by default. It is a genuine speed-up and it introduces a window in which acknowledged writes can be lost on a crash. Acceptable for logs where the source can replace; unacceptable for anything else.

Segments and merging

- Every refresh creates a segment. Many small segments slow search, because each is consulted separately
- Merging runs continuously in the background, combining small segments and physically dropping deleted documents
- Merging competes with indexing and search for I/O — a heavy ingest period will show elevated merge activity and latency
- Deleted documents occupy space until merged. A large delete or update rate means a persistently inflated index

Force merges is for read-only indices only. Performing a finished time-based index down to one segment is a genuine and worthwhile optimisation. Running it on an index still receiving writes produces very large reads that force merges struggle with, and the effect is long-lived. The rule is simple: only force-merge an index that will not be written to again.

Rejections and backpressure

- Write threat pools have bounded queues. When ML requests are rejected — this is deliberate backpressure, not a failure to fix by enlarging the queue
- Rejections must be relied with backoff by the client. Most official clients and Logstash do this; hand-written ingest often does not, and silently drops data
- A rising rejection count means the cluster cannot keep up — the answer is more nodes, better storage, or less work per document, not a bigger queue

Check bulk responses per item. A bulk request returns HTTP 200 while individual items failed. Code that checks only the status code loses documents silently — one of the most common and least visible ingest bugs in production.

6 · SHARD SIZING

the number to get right first

Primary shard count is fixed when the index is created. It cannot be changed in place, but the shard for a document is derived from a hash of its id. Splitting and shrinking exist, and both write a new index. In practice, plan to — or use rollover for each new index gets a fresh decision.

The guidance, and what it is based on

10-50 GB per shard: The widely used working range. Smaller wastes overhead; much larger slows recovery, relocation and merges

Around 20 shards: Per node. A node with 30 GB of heap should stay under roughly 600 shards, and comfortably below that in practice

Shards < nodes: Fewer shards than data nodes leaves nodes idle for that index

4 worked examples: 2 TB of log data, 30-day retention, daily volume ~ 2 TB / 30 ~ 66 GB/day, target shard size ~ 40 GB, primaries per day ~ 66 / 40 ~ 2, with 1 replica ~ 4 shards/day, over 30 days ~ 120 shards total

the wrong instinct ~ 5 primaries per day “for speed” ~ 300 shards of ~14 GB each, more overhead, more merging, slower recovery, not faster

Overhauling is the classic mistake, and it is usually well-intentioned. More shards feels like more parallelism. But each shard is a full Lucene index with fixed overhead, and every search must coordinate across all of them. Past the point where shards are large enough to be efficient, adding shards makes the cluster slower and less stable, not faster.

For time-based data sets, use data streams with rollover rather than deciding a shard count once and living with it. Rollover creates a new backing index at a size or age threshold, so each index is sized against actual volume — retention becomes dropping while indices, which is nearly free, instead of deleting documents, which is expensive.

7 · QUERYING AND RELEVANCE

The query types worth knowing

Query	Behaviour
match	Analyses the input, then looks for the resulting terms. The default choice for full-text
term	Does not analyse. Exact term lookup — correct on keyword id, and a common source of empty results on text
match_phrase	Terms in order and adjacent
multi_match	One query across several fields, with per-field weighting
bool	Combines clauses: must, should, filter, must_not. The workhorse
range	Numeric, date and other ordered types. Almost always belongs in filter context
ids	Expensive. A leading wildcard will use the index at all — use edge_ngram instead
vector_score / rbrqrr	Vector similarity search over dense vectors — semantic search and retrieval for language models

The optimisation to apply everywhere: move yes-or-no conditions into filter. They then skip scoring entirely and their results are cached and reused across queries. A status, a tenant id, a date window and a category all belong there. This is frequently the largest single improvement available to a slow search, and it changes no behaviour.

How relevance is calculated

- BM25 is the default similarity. Three intuitions drive it: a term appearing more often in a document scores higher, a term appearing in fewer documents overall is more discriminating and worth more, and a match in a shorter field counts for more than the same match in a long one
- Scores are relative to one query and are not comparable between queries or across indices. A score of 8 means nothing on its own
- explain shows the full scoring breakdown for a document — the way to answer “why is this result ranked third?”
- Boosting weights fields or clauses: function_score and rank_feature fold in signals like recency or popularity

STAFF: Why scores can look wrong on a small index. Term statistics are per shard, not cluster-wide. With few documents unevenly distributed, the same term can effect differently depending on which shard holds it, and results appear mis-ranked. dfs_query_then_fetch gathers global statistics first and fixes it, at the cost of an extra round trip. The effect disappears naturally as documents accumulate — so it is mostly a test-environment surprise.

Getting relevance right is an iterative, measured activity, not a configuration setting. Build a judgement set of queries with expected results, change one thing, and measure. Tuning boosts by intuition until a few examples look right really makes other queries worse without anyone noticing.

Modern retrieval combines both. Keyword search via BM25 and vector search via knn have complementary failure modes — lexical matching misses paraphrase, semantic matching misses exact identifiers. Hybrid retrieval that combines both rankings is the current practice for search building language-model applications.

8 · AGGREGATIONS

and their approximations

Metric	Purpose
Count	A value over a set — avg, sum, min, max, percentile, cardinality
Bucket	Groups documents — terms, date_histogram, range, filters, nested
Pipeline	Operates on the output of other aggregations — moving averages, derivatives, bucket selection

Two results are approximate, and this genuinely surprises people. cardinality uses HyperLogLog — a distinct count that is close but not exact, in bounded memory. terms returns counts that may be slightly wrong, because each shard returns only its own top buckets and a term just outside one shard’s cut-off is under-counted. The response reports doc_count_error_upper_bound, and it is almost universally ignored.

Rollup shard_size above 1 size when accuracy matters. Each shard then returns more candidates than you need, which shrinks the error at the cost of memory and transfer. Exact counts require either one shard or a different tool.

The mechanism, and the memory limits

- Aggregations read doc_values — a column, on-disk structure held in the filesystem cache rather than on the heap. Enabled by default for non-text fields
- Aggregating a text field requires fieldsets, which is an on-heap structure and disabled by default for good reason. Aggregate a keyword instead — enabling fieldsets does make an aggregation work as how nodes run out of heap
- There is a cap on total buckets per request, defaulting to 65,536. A terms aggregation nested into a date_histogram multiplies buckets and reaches it quickly
- The cap is a protection, not an obstacle. Raising it to make a dashboard load means the failure from a clear error to an out-of-memory event

For repeated aggregations over unchanging history, use a transform to pre-compute results into a summary index. Aggregating the same billion documents on every dashboard refresh is work that only needs doing once.

9 · CLUSTER & HIGH AVAILABILITY

Getting the topology right

- Three dedicated master-eligible nodes, in separate failure domains. They need little CPU or disk and they keep the cluster stable — a master that is also a busy data node is the classic cause of instability
- At least one replica for every index you cannot afford to lose or to have unavailable
- Dedicated coordinating nodes in front of heavy search or kibana traffic, so useful merging does not consume data-node heap
- Shard allocation awareness so primary and replica never land in the same rack or zone. Without it, replicas are not protecting you from the failure you actually expect
- Separate ingest and ML roles at scale, so a heavy pipeline cannot starve search

A yellow cluster is not an emergency; a red one is. Yellow means a replica is unassigned — everything works, redundancy is reduced. Red means a primary is unassigned and some data is unavailable right now. Alerting that treats them the same produces noise that hides the case that matters.

10 · LIFECYCLE & DATA TIERS

Time-series data — logs, metrics, events — is written once, queried heavily when new, and rarely afterwards. Index lifecycle management moves indices through tiers matched to that cycle, so old data does not sit on expensive hardware.

Tier: Hardware and role

- Hot: Actively written and queried. Fast local SSD or NVMe, the most expensive nodes
- Warm: No longer written, still queried. Cheaper storage, often lower replicas
- Cold: Rarely queried. Can be backed by searchable snapshots, removing the need for a full local replica
- Frozen: Queried very rarely. Data stays in object storage and is fetched on demand — dramatically cheaper, and much slower

The phases of a policy

- Rollover — start a new index at a size, age or document threshold. The foundation of the whole approach
- Shrink — reduce the shard count once writing has stopped
- Force merge — down to one segment, now safe because the index is read-only
- Searchable snapshot — move the data to object storage while keeping it queryable
- Delete — drop the index at end of retention

Deleting an index is nearly free; deleting documents is expensive. A dislatch by query marks every matched document and forces merge to reclaim the space. Dropping a whole index removes files. This is the strongest argument for time-based indices — retention becomes a deletion rather than a large background job.

Use data streams for append-only time-series data. They wrap rollover, naming and backing-index management behind one name you write to, and are the current recommended approach — rather than hand-rolled date-aware index names and a cron job to delete them.

Component templates compose index into templates, so mappings and settings shared across many indices are defined once. Combined with data streams this makes a new data source a small configuration change rather than a new bespoke pipeline.

11 · SECURITY

the defaults are not enough

An unsecured Elasticsearch on a reachable network is fully readable and writable by anyone who finds it. The API is HTTP and has no authentication until you enable it. Publicly exposed clusters have been a recurring, large-scale source of data breaches, and they are scanned for continuously.

The layers

- Network: Bind to private interfaces. Never expose the HTTP port publicly, regardless of authentication. Firewall the transport port separately
- TLS: On both the HTTP layer and the transport layer between nodes. Transport TLS is what protects your unauthorised nodes joining the cluster
- Authentication: Native realm, LDAP, Active Directory, SAML, OAuth, Certificates, PKI — and API keys for services
- Authorization: Role-based, down to index and action. Built-in and custom roles
- Field and document level: Restrict which fields a role sees, and which documents — the mechanism for multi-tenancy within one index
- Audit logging: Records access and changes. Required in most regulated environments

Practices worth insisting on

- API keys per service, scoped and rotatable — not a shared superuser password in a configuration file
- Least privilege: a dashboard needs read on specific indices, not cluster admin
- Kibana is a separate attack surface and needs its own authentication and network controls
- Reusable dynamic scripting where it is not needed, and keep private access restricted
- Encrypt snapshots at rest — the repository holds a full copy of everything

Field-level security is not a substitute for separate indices when tenants are genuinely untrusted. It is enforced correctly, and it puts one misconfigured rule between one customer’s data and another’s. Weigh that against the operational cost of separation deliberately.

12 · MEMORY, THE JVM AND CIRCUIT BREAKERS

Sizing the heap

No more than 50% of RAM: The rest is for the filesystem cache, which Lucene depends on completely

Stay below the compressed pointer threshold: 28 GB is safe on essentially all systems, and some can go to around 30 GB. Above it the JVM abandons compressed object pointers and effective capacity falls — more heap yields less usable memory

Xms & Xmx: Avoid resizing pauses. Set both to the same value

Lock memory: Prevent the heap from being swapped. A swapping node is catastrophically slow and looks like a network problem

The commonly repeated version of this rule is “50% of RAM, max 32 GB”, and it is imprecise. The threshold where compressed pointers stop applying sits below 32 GB, and the documented safe figure is 28 GB. A cluster configured with 32 GB of heap may be performing worse than same compressed pointer assumption — the node into API reports whether compressed pointers are in use, and the answer should be yes.

The help you did not get to the heap is doing the most work. Lucene reads its index files through the operating system’s page cache, so the filesystem cache is what makes search fast. A node with 64 GB of RAM and 28 GB of heap has around 36 GB caching index data — and that is usually the better allocation. Raising the heap to 48 GB actually harms search performance.

Circuit breakers

Elasticsearch estimates the memory an operation will need and rejects it before allocation if it would breach a limit. A rejected request is far better than a node lost to an out-of-memory error, which takes its shards with it.

Breaker	Guards against
Parent	Total across all breakers — the backstop
Field data	Loading from field data into the heap. Usually triggered by aggregating a text field
Request	A single request’s structure — most often a very large aggregation
In-flight requests	The total size of requests being processed — oversized bulk bodies

Test a tripped breaker as a diagnosis, not an obstacle. It names the query or the bulk request that would have taken the node down. Raising the limit removes the protection and leaves the case in place; the fix is the aggregation, the field being aggregated, or the batch size.

Reading the memory picture

- Heap sawtooth between roughly 50% and 70% is healthy — allocation and collection working normally
- Heap that stays above 85% means repeated full collections and long pauses. The node may be evicted from the cluster for being unresponsive
- Long garbage-collection pauses are a leading indicator of instability — track pause duration, not just heap percentage
- The most common causes: too many shards per node, aggregations over very high cardinality, deep pagination, and oversized bulk requests

13 · SNAPSHOTS & RECOVERY

replicas are not a backup

Replicas protect against hardware failure and nothing else. A mistaken delete, a bad mapping change or a faulty ingest pipeline is replicated faithfully and instantly to every copy. Only a snapshot recovers from those — and they are the failures that actually happen.

How snapshots work

- Incremental at the segment level. Because segments are immutable, a snapshot copies only segments not already in the repository — so after the first, subsequent snapshots are small and fast
- The repository is external — object storage or a shared filesystem. It must be reachable by every data node
- They do not block indexing: a snapshot is a consistent point in time rather than a write freeze
- Restore is per index, and you can restore on restore — which makes recovering one index into a live cluster straightforward
- Automate the schedule with a snapshot lifecycle policy, including retention

The parts people miss

- Test a restore, and time it. An untested backup is a hypothesis, and restore duration is a number your recovery objective depends on
- Keep the repository in a different failure domain from the cluster
- Snapshot the cluster state too, so templates, ILM policies and roles come back with the data
- Retention must be automated, or the repository grows until it becomes a cost problem nobody owns
- Encrypt at rest — the repository holds a full copy of everything

The genuinely strong position: be able to rebuild from source. If the data can be re-indexed from a database or an event log, a total cluster loss is a slow day rather than a disaster. Snapshots then reduce recovery time instead of being the only thing standing between you and permanent loss — and that is a much better place to be.

The settings that change behaviour, what the application must get right, and what to watch

Metric	Why
Cluster status	Red is data unavailable; yellow is reduced redundancy. Alert differently
Unassigned shards	Persisting merge allocation is failing — the explain API gives the reason
JVM heap and GC	Sustained heap above 85%, or long pauses, precede node instability
Threat pool rejections	Write and search queues overflowing — the cluster cannot keep up
Circuit breaker trips	Names the query or batch that would have taken a node down
Search and index latency	Per index, at high percentiles rather than the mean
Disk usage vs watermarks	Alert well below the low watermark — food stage makes indices read-only
Shard count per node	Against the heap-based guideline. Creeps upward unnoticed
Segment count and merge time	Merge pressure from heavy indexing or updates
Snapshot success	A silently failing snapshot means no backups

The tools: cluster/health_and_cat/indices_and_cat/shards for state, cluster/allocation/explain for why a shard will not assign, nodes/stats for heap and thread pools; the slow log per index for queries over a threshold, and tasks to find and cancel a runaway search.

15 · PRODUCTION FAILURES — SYMPTOM, CAUSE AND FIX

Symptom: Cause: Fix

Nodes leaving, heap constantly high: Too many shards per node, or heavy aggregations. Reduce shard count; check the heap-per-shard ratio; fix the aggregation

Search returns nothing, no error: Analyser mismatch, or a term query against an analysed field. _analyze to compare tokens; use test-oh query; keyword sub-field. Do not enable fieldsets

Aggregation or sort fails on a field: It is mapped as text; doc_values are unavailable. Use the keyword sub-field. Do not enable fieldsets

Writes rejected, cluster looks fine: Disk-flood stage set indices read-only. Free space, then release the read-only block explicitly — it does not clear itself

Cluster state huge, everything slow: Mapping explosion from dynamic fields on user-controlled keys. Remodel as key/value pairs or flattened; set dynamic: strict

Symptom: Cause: Fix

One query takes the cluster down: Deep pagination, an unbounded aggregation, or a leading wildcard. Merge pressure, slow disk, or oversized bulk requests

Indexing throughput collapses: Documents missing after ingest. Per-item bulk failures never inspected. Allocation filtering, disk watermark, or no matching tier

Shards will not assign: Mappings are fixed once written. Mappings are fixed once written

Cannot change a field type: Per-shard term statistics with few documents

Relevance looks wrong on a small index: Relevance looks wrong on a small index

What breaks first

#	Limit
1	JVM heap, usually from too many shards or an unbounded aggregation. By a wide margin the most common cause of instability
2	Shard count per node — creeps up quietly and degrades everything at once
3	Disk watermarks, which stop writes in a way that looks like a healthy cluster
4	A single expensive query — deep paging or a huge aggregation taking down a node
5	Raw indexing throughput — the calling people plan for, and the last one reached

Note the order. The limit stages are built around — documents per second — is last. The ones that cause real incidents are memory and shard hygiene, both of which are visible in dashboard weeks in advance if anyone is looking.

Degradation

- Cluster unavailable, used for search — the application should degrade to a database query or a cached result, not return an error page
- Cluster unavailable, used for logging — buffer at the shipper and backfill. Never block the application on log delivery
- Partial results — a search across many shards can return successfully with some shards failed. Check the shard failure count in the response, or you will silently show incomplete data

16 · PRODUCTION CHECKLIST

□ Data is rebuildable from a source of truth elsewhere

□ Every field explicitly mapped: dynamic: strict under indexes

□ Text versus keyword decided per field, deliberately

□ No user-controlled field names — key-value or flattened matters

□ At least one replica on every index

□ Field count monitored against the per-index limit

□ Analysers tested with _analyze at index and query time

□ Applications read and write through aliases, never index names

□ Refresh procedure written down and rehearsed

□ Shard size in the 10-50 GB range

□ Shards per node within the heap-based guideline

□ Not overhauling — shard count justified by volume, not instinct

□ Time-based data uses data streams with rollover

□ ILM policy defined with tiers and a delete phase

□ Retention deletes indices, not documents

□ Index templates and component templates in place

□ Three dedicated master-eligible nodes, separate failure domains

□ At least one replica on every index

□ Allocation awareness configured so copies span zones

□ Dedicated coordinating nodes in front of heavy search

□ Heap < 50% of RAM and below the compressed-pointer threshold

□ Compressed pointers verified as in use, not assumed

□ Test equals Xmx; memory tested; swap off

□ Circuit breaker trips alerted on, not raised away

□ Bulk API used, sized by measurement

□ Every bulk response item checked for per-item failures

□ Rejections relied with backoff by the client

□ Translog durability left at the default unless loss is acceptable

□ force_merge only on read-only indices

□ Yes-or-no conditions in filter context

□ No deep pagination — search_after with a point-in-time

□ Aggregation bucket counts bounded, no fields on the

□ Approximate results understood — cardinality and terms counts

□ Shard failure count checked in every search response

□ Slow log enabled and reviewed regularly

□ Security enabled, TLS on for HTTP and transport

□ Never exposed to the public internet

□ API keys per service, least privilege, rotatable

□ Snapshots automated to an external repository, with retention

□ A restore tested and timed, including cluster state

□ Behaviour defined for the cluster being unavailable

The one that summarises the sheet: state how large a shard is and how many exist, whether every field is explicitly mapped, whether applications read through an alias, and how long it would take to rebuild the cluster from source. Those four answers separate a search layer that can be operated and evolved from one that will need an outage to change — and the last is the question worth answering before an incident rather than during one.